



Penetration Testing

Gala (Launchpad)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Gala team partnered with Hashlock to conduct a security Penetration Test of their Gala Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Gala is building a decentralized entertainment ecosystem that spans gaming, music, film, and blockchain technology. Their platform, GalaChain, is an L1 blockchain optimized for entertainment, offering fast transaction speeds, scalability, and energy efficiency. Gala provides developers with tools like Gala Creators and GalaChain Connect to create and deploy Web3 applications seamlessly. The ecosystem supports various projects, including games like Champions Arena and Spider Tanks, Gala Music for blockchain-based music distribution, and Gala Film for decentralized cinema experiences. Gala's mission is to empower creators and players by providing them with real ownership and control over their digital assets.

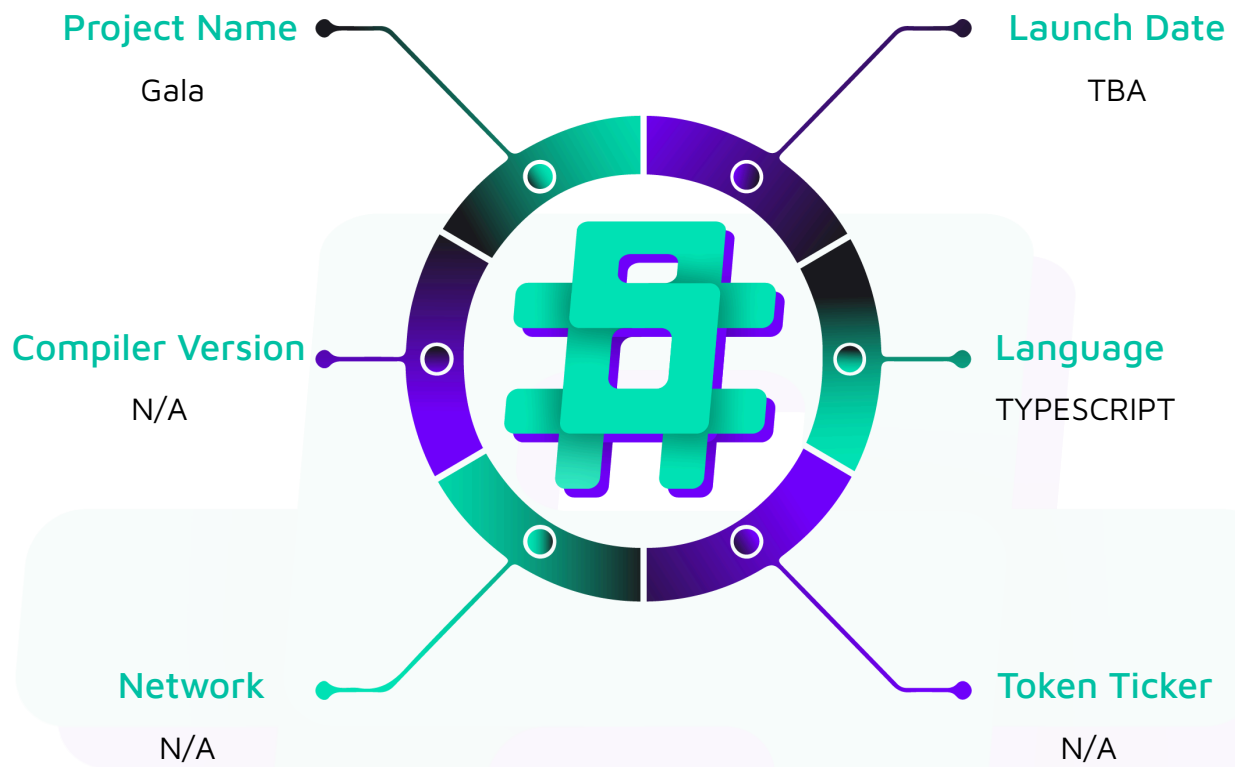
Project Name: Gala

Project Type: Wallet

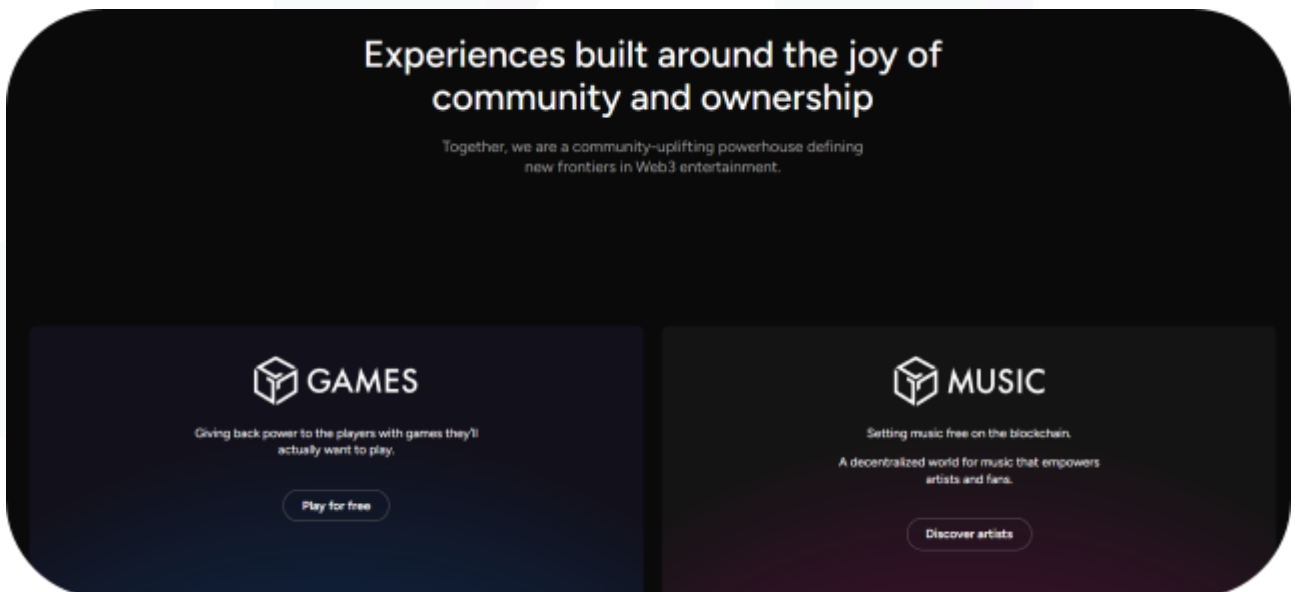
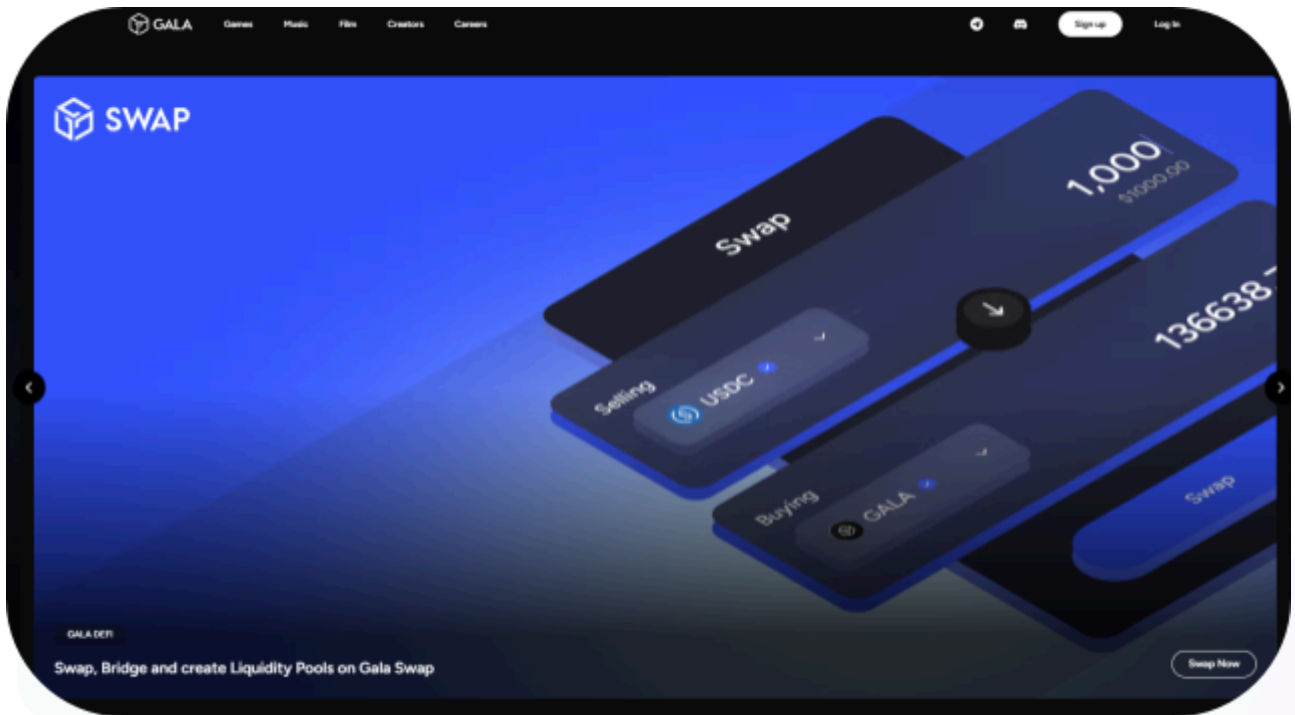
Website: gala.com

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Gala project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Gala Project code base
Platform	Web apps & APIs
Penetration Test Date	November, 2025
Repository/Website URL/IP Tested	https://github.com/GalaChain/launchpad/tree/main/src
GitHub Commit Hash (If applicable)	e368346df8bc0463e3a9eb480d4e702ccde95365
Fix Review GitHub Commit Hash (If applicable)	76ad3a41332dd7ffdc6dd3a8998d5a1c7d08c8e7

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 Medium-severity vulnerabilities

1 Low-severity vulnerabilities

4 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Medium

[M-01] buyWithNative#Token Purchase - Market cap enforcement mismatch allows user overpayment

Description

The `buyWithNative` function enforces market cap limits during price calculation but uses the original uncapped native token amount for transfers and state updates, allowing users to pay more than the priced amount and breaking the sale's economic invariant.

Vulnerability Details

The `callMemeTokenOut` function correctly caps native token input when calculating token amounts, but `buyWithNative` transfers the original `buyTokenDT0.nativeTokenQuantity` and updates the sale state with this uncapped value. This discrepancy causes buyers near the market cap to overpay, with excess funds retained in the vault until finalization.

```
// callMemeTokenOut.ts - Correctly caps native tokens for pricing

if      (nativeTokens.add(new      Decimal(sale.nativeTokenQuantity)).greaterThan(new
Decimal(LaunchpadSale.MARKET_CAP))) {

    nativeTokens      =      new      Decimal(LaunchpadSale.MARKET_CAP).minus(new
Decimal(sale.nativeTokenQuantity));

}

// buyWithNative.ts - Uses uncapped amount for transfers and state updates

await transferToken(ctx, {

    from: ctx.callingUser,

    to: buyTokenDT0.vaultAddress,

    tokenInstanceKey: nativeToken,
```

```
quantity: buyTokenDTO.nativeTokenQuantity, // Uncapped amount

allowancesToUse: [],

authorizedOnBehalf: undefined

});

sale.buyToken(tokensToBuy, buyTokenDTO.nativeTokenQuantity); // Uncapped amount
```

Impact

Users may pay more native tokens than priced for their purchase.

Recommendation

Return the effective capped native amount from `callMemeTokenOut` and use this value for all transfers, balance checks, and state updates in `buyWithNative`.

Status

Resolved

[M-02] Pricing#Bonding Curve - Inconsistent base price source enables arbitrage

Description

Different pricing functions use inconsistent base price sources, allowing users to exploit price discrepancies between trading paths.

Vulnerability Details

The `callMemeTokenOut` function uses a hardcoded `LaunchpadSale.BASE_PRICE` while `callNativeTokenIn` uses `sale.fetchBasePrice()`. If a sale configures a custom base price, this creates pricing arbitrage opportunities between buy paths.

```
// callMemeTokenOut.ts - Hardcoded base price
const basePrice = new Decimal(LaunchpadSale.BASE_PRICE);

// callNativeTokenIn.ts - Sale-specific base price
const basePrice = new Decimal(sale.fetchBasePrice());
```

Impact

This may lead to price discrepancies between trading paths, allowing potential arbitrage or value extraction.

Recommendation

Standardize all pricing functions to use `sale.fetchBasePrice()` and add runtime assertions to ensure consistent pricing inputs across all calculation paths.

Status

Resolved

[M-03] buyWithNative#Fee Calculation - Stale transaction fee after vault capacity adjustment

Description

When `buyWithNative` adjusts the purchase due to insufficient vault tokens, it recalculates native token requirements but retains the original transaction fee, potentially overcharging users.

Vulnerability Details

After capping the token purchase to vault capacity, the function recalculates native token requirements but does not update the transaction fee, using the stale fee from the initial calculation for balance checks and transfers.

```
if (tokensLeftInVault.comparedTo(tokensToBuy) <= 0) {  
    tokensToBuy = tokensLeftInVault.decimalPlaces(memeTokenClass.decimals);  
  
    const callNativeTokenInResult = await callNativeTokenIn(ctx,  
nativeTokensRequiredToBuyDto);  
  
    transactionFees = callMemeTokenOutResult.extraFees.transactionFees; // Stale fee  
  
    buyTokenDTO.nativeTokenQuantity = new  
    BigNumber(callNativeTokenInResult.calculatedQuantity);  
  
    isSaleFinalized = true;}
```

Impact

Users may be charged incorrect transaction fees that don't correspond to their actual purchase size.

Recommendation

Update `transactionFees` to use `callNativeTokenInResult.extraFees.transactionFees` after recalculating the native token requirements for capped purchases.

Status

Resolved

Low

[L-01] Fee Transfer#Transaction Processing - Zero-value fee transfers waste gas

Description

The transaction fee check uses `if (launchpadFeeAddressConfiguration && transactionFees)` where `transactionFees` is a string, causing the condition to pass for "0" values and attempt zero-value token transfers. This causes unnecessary gas consumption from zero-value transfer attempts.

Recommendation

Replace with numerical comparison: `if (launchpadFeeAddressConfiguration && new BigNumber(transactionFees).gt(0))`

Status

Resolved

QA

[Q-01] CLI - Incompatibility with Windows OS

Description

The Command-Line Interface (CLI) utility script located at `src/cli.ts` contains a hardcoded file path for its output. Specifically, the `get-contract-api` command is configured to always write its output to `/tmp/contract-api.json`. The `/tmp` directory is a standard convention on Unix-like operating systems (e.g., Linux, macOS) but is not present on native Windows environments.

Recommendation

Refactor the script to use a platform-agnostic method for determining the system's temporary directory. In Node.js, the recommended approach is to use the `os.tmpdir()` function from the built-in `os` module.

Status

Resolved

[Q-02] Launchpad#configureLaunchpadFeeAddress - Incorrect value check

Description

The transaction fee check uses `if (!dto.newPlatformFeeAddress && !dto.newAuthorities?.length && !dto.newFeeAmount)` where `newFeeAmount` is a number, since number `0` is falsy it is impossible to set the fee to `0`.

Recommendation

The conditions should check whether the `dto.newFeeAmount` value is undefined.

Status

Resolved

[Q-03] Launchpad#LaunchpadContract - Hardcoded Org names**Description**

While some `allowedOrgs` values use the organization name given by environment variables some of them just use the hardcoded values : `allowedOrgs: ["CuratorOrg"]`

Recommendation

Hardcoded organization names should be changed to `allowedOrgs: [process.env.CURATOR\_ORG\_MSP ?? "CuratorOrg"]`

Status

Resolved

[Q-04] Launchpad#launchpadBatchSubmitAuthorizations.spec.ts - Incorrect**Test Setup****Description**

The way the test is currently set up to give `allowedOrgs` does not align with other tests. When used this way any organization name seems to pass, whether it's in allowed orgs or not.

Recommendation

Test context should be fixed to use `caClientIdentity`

Status

Resolved

Conclusion

After Hashlock's analysis, the Gala project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.